

Qué implementar para un tracking de remediación sólido

Nota de alcance

Este documento reemplaza al análisis de brechas anterior (`Brechas_Tracking_Remediacion.md`) con una versión **recortada a lo estrictamente necesario**. Se sacó todo lo que, según el propio marco de referencia (`Tracking_Remediacion_Documento_Unificado.md`), el marco de `caja_negra_caja_blanca_ciberseguridad.md` , y la conversación previa, es mejora de madurez avanzada, depende de una segunda fuente de detección que hoy no existe, o es un ejercicio de gestión más que una carencia técnica.

Implementar únicamente lo que sigue alcanza el **nivel 3 de la escala de madurez de verificación** (Sección 10.1 del marco) — el punto en el que el sistema verifica por sí mismo en lugar de confiar en declaraciones humanas, y que el propio marco define como el salto cualitativo que separa un gestor de tareas de un tracking de remediación real.

Cada punto indica si depende solo del backend de este repositorio o si requiere un cambio en el pipeline externo de ingestión (n8n).

1. Fundamento del modelo — sin esto no hay tracking en sentido estricto

1.1 Registro de auditoría inmutable de transiciones (Componente F)

Cada cambio de estado debe generar un evento propio (estado anterior, estado nuevo, timestamp, actor, tipo de disparador) en lugar de sobrescribir el campo actual. Hoy `VulnerabilityAuditService.updateStatus()` pisa `triageStatus` y pierde `resolvedAt` cuando el estado cambia. Sin esto no hay tiempo de permanencia por estado, no hay forma de recalcular métricas históricas, y no hay trazabilidad ante una auditoría.

Alcance: backend.

1.2 Cobertura de escaneo declarada antes de inferir cierre (Componente A)

`applyLifecycle()` marca `RESOLVED` a todo hallazgo que no reaparece en la corrida actual, sin verificar si el activo fue efectivamente analizado. Hace falta:

- Que `ScanReport` registre `assets_in_scope` , `assets_analyzed` y use realmente los estados `PARTIALLY_COMPLETED` / `FAILED` (hoy siempre se setea `COMPLETED`).
- Que `applyLifecycle()` consulte esa cobertura antes de inferir remediación por ausencia, y que la inferencia quede registrada como tal (no como un cierre indistinguible de uno verificado).

Es el punto que el marco llama "*el error más peligroso de un tracking mal construido*": sin esto, las métricas mejoran cuando en realidad empeora la cobertura del escaneo.

Alcance: backend.

1.3 Corregir la etiqueta de MTTR

El intervalo que hoy se llama `mttrDays` termina en una declaración humana (`RESUELTA`), no en una verificación. Una vez resuelto 1.2, el sistema puede calcular un MTTR real (fin = verificación automática) y debe exponer ambos por separado: tiempo de gestión (declarado) y MTTR (verificado). No renombrar el dato es el antipatrón "MTTR nominal" que el marco señala explícitamente.

Alcance: backend.

1.4 Estados de evidencia completos (vocabulario VEX)

Ampliar `triageStatus` más allá de `DETECTADA` / `EN_ANALISIS` / `RESUELTA` para incluir:

- **Mitigada** — riesgo reducido por una vía distinta a remediar (control compensatorio, aislamiento), con descripción del control.
- **No aplica** — con justificación obligatoria (no texto libre opcional como hoy).
- **Riesgo aceptado** — con responsable y **fecha de caducidad obligatoria**. Sin vencimiento, una aceptación de riesgo se vuelve permanente por omisión — es un antipatrón directo del marco.

Sin estos estados, remediar y mitigar quedan indistinguibles del "sin tratar", que es exactamente la pérdida de información que el marco advierte.

Alcance: backend.

2. Priorización y evidencia — sin esto el plazo y el cierre no son defendibles

2.1 Plazo (SLA / fecha límite) derivado de un criterio de priorización (Componente C)

No existe hoy ningún campo de fecha límite. Hace falta un campo calculado a partir de la clasificación de riesgo del caso (severidad + contexto del activo, no solo CVSS), documentado y auditable. Sin esto no hay "cumplimiento de SLA" que medir — es un número que no existe, no uno arbitrario.

Alcance: backend.

2.2 Ampliar el criterio de priorización más allá del CVSS puro

`priority` se propaga hoy directamente del hallazgo del escáner, sin incorporar exposición del activo (producción vs. desarrollo, expuesto a Internet o no) más allá del booleano `exploited`. No hace falta adoptar el modelo completo de la BOD 26-04 (eso queda en la sección de "no necesario"), pero el diseño debe dejar espacio para que la exposición del activo module la prioridad, tal como ya lo anticipa el propio flujo operativo del marco (Sección 6.3).

Esto es más barato de cerrar de lo que parece: `Environment.businessCriticality` (`Environment.java:19-20`) **ya existe en el modelo y ya se carga por seed** (`Testing` → `MEDIA` , `Producción` → `CRITICA`), pero no se usa en ningún cálculo de prioridad — es un campo cargado y no leído. Incorporarlo al cálculo de `priority` cierra buena parte de esta brecha sin modelar nada nuevo.

Alcance: backend.

2.3 Escala de evidencia y captura de evidencia de respaldo (E0–E6)

Registrar qué nivel de evidencia respalda cada cierre (declaración manual, ticket, despliegue con timestamp, inventario posterior, reevaluación independiente) y permitir adjuntar o referenciar el respaldo (reporte de escáner, commit, PR). Sin esto, un cierre por declaración y un cierre por inferencia por ausencia tienen la misma fuerza probatoria nominal, y no se puede distinguir una métrica de actividad de una métrica de resultado verificado.

Alcance: backend.

2.4 GHSA estructurado: rango de versiones, `reviewed` / `unreviewed` , `withdrawn`

`GhsaAdvisoryCache` guarda hoy solo texto descriptivo. Para que la verificación (1.2/1.3) sea confiable hace falta que el sistema conozca el rango de versiones afectado, si el advisory está `withdrawn`, y si es `reviewed` o `unreviewed` (un advisory `unreviewed` con rango placeholder puede producir un falso cierre). Esto **requiere que el pipeline de n8n empiece a entregar esos campos** — no es algo que el backend pueda resolver ingiriendo solo lo que ya recibe hoy.

Alcance: backend + pipeline externo (n8n). Es el único punto de esta lista que no se puede cerrar solo con cambios en este repositorio.

3. Métricas y estructura de datos — consecuencia directa de lo anterior

3.1 Exponer los KPIs que ya quedan habilitados al resolver 1–2

Una vez resuelto lo anterior, agregar a `DashboardService` :

- MTTR real vs. tiempo de gestión (separados, ver 1.3)
- Tiempo medio de reconocimiento (detección → primer triage humano)
- MTTV (declaración → verificación)
- Tasa de eficacia de remediación (declaradas vs. verificadas)
- Cumplimiento de SLA (bloqueado hasta 2.1)
- Tiempo de permanencia por estado (bloqueado hasta 1.1)
- Cobertura del escaneo (bloqueado hasta 1.2)
- Tasa de recurrencia — el dato (`reopenCount`) ya existe, solo falta agregarlo y exponerlo en `DashboardStatsDTO`

Alcance: backend. Es trabajo mecánico una vez que los componentes de base existen — no requiere decisiones de diseño nuevas.

3.2 Separar el hallazgo persistente del ciclo de remediación

Hoy `AssetVulnerability` mezcla ambos: al reabrirse, sobrescribe `detectionStatus` / `triageStatus` en la misma fila en vez de crear un nuevo ciclo con sus propios timestamps. Esto conviene resolverlo junto con 1.1 (son la misma decisión de modelado: eventos como entidad de primer orden), porque hacerlo después obliga a rehacer las consultas de métricas ya construidas.

Alcance: backend.

3.3 Asignación estructurada de responsable

`assignedTo` es hoy texto libre. Conviene normalizarlo a una relación con `User` /equipo para que el KPI "vulnerabilidades por responsable" y el SLA por caso sean confiables. Es de menor urgencia que 1–2, pero necesario para que el catálogo de KPIs sea completo y no dependa de texto sin normalizar.

Alcance: backend.

4. Cobertura de detección: el sistema es hoy 100% Caja Blanca

`caja_negra_caja_blanca_ciberseguridad.md` describe un modelo combinado (Sección 5 de ese documento): escaneo externo (Black Box, "qué está expuesto") + agente interno (White Box, "qué software existe realmente"), correlacionados.

Revisando el pipeline real (`InventoryService` , `PurlParser` , `GhsaAdvisoryService` , `ScanService`), el sistema hoy **solo implementa el lado White Box**: recibe inventario de dependencias vía `purl` (SBOM/CycloneDX)

y lo correlaciona contra GHSA. No existe ningún componente de descubrimiento externo — ni en este backend ni evidencia de que el pipeline de n8n lo haga: no hay campos de puerto, servicio, banner, ni un `targetType` que represente "host escaneado desde afuera". Confirmado por búsqueda dirigida (`nmap` , `puerto` , `portScan` , `banner` , `shodan`) sin resultados en todo el backend.

Por qué esto no es una brecha de "tracking sólido": el marco de remediación

(`Tracking_Remediacion_Documento_Unificado.md` , Sección 6.1) lista el escaneo externo como *una fuente de detección entre varias* — el tracking no necesita generar sus propias detecciones, necesita gestionar bien las que recibe. Agregar Black Box real sería agregar una **segunda fuente de detección**, lo cual activa el mismo problema de deduplicación multi-fuente que ya quedó marcado como "no necesario hoy" en la Sección 5 más abajo, porque hoy solo hay una fuente.

Qué sí es relevante ahora: que el campo de exposición del activo (usado en 2.2 para priorizar) sea un dato **declarado y auditable** — `businessCriticality` del `Environment` , o un campo propio a nivel `Asset` si se necesita granularidad por activo — y no algo que el sistema deba descubrir por sí mismo escaneando la red. Eso ya está cubierto por el punto 2.2 sin necesidad de Black Box.

No se agrega como ítem de implementación: es una aclaración de alcance, no una tarea pendiente.

5. Lo que se saca — no es necesario para un tracking sólido

- **Verificación nivel 4 (reescaneo activo sobre el sistema en ejecución).** El propio marco aclara que la mayoría de los proyectos no lo necesita; el nivel 3 (verificación por inventario) ya es suficiente.
 - **Deduplicación entre múltiples fuentes de detección.** Hoy hay una única fuente (n8n). Es un problema real solo si se agrega una segunda fuente de detección — no antes.
 - **Integración completa de KEV / EPSS / SSVc.** Mejora de madurez sobre la priorización (2.2 ya deja espacio para incorporarlo después). La BOD 26-04 es vinculante solo para agencias federales de EE. UU.; acá es referencia, no obligación.
 - **Autoevaluación formal contra el modelo de madurez SANS.** Es un ejercicio de gestión que solo tiene sentido una vez cerradas las secciones 1–3 — evaluarlo antes solo confirmaría lo ya evidente.
 - **Anclaje normativo explícito a la Decisión Administrativa 641/2021 (Argentina).** Relevante solo si el sistema necesita demostrar alineación normativa local ante una auditoría o un cliente del sector público — no es una carencia funcional del tracking en sí.
 - **location_key (ubicación dentro del activo).** La identidad actual (`componente` , `CVE`) es suficiente mientras la granularidad de detección sea "un CVE por componente". Se vuelve necesario recién si en algún momento el mismo CVE puede aplicar a más de una instancia dentro del mismo componente — no es el caso hoy.
 - **Escaneo Black Box (descubrimiento externo de puertos/servicios).** Ver Sección 4. Es una fuente de detección adicional, no un requisito del tracking — y agregarla activaría de inmediato el punto de deduplicación multi-fuente de arriba.
-

6. Una salvedad que no es código

La separación de funciones en la verificación (que quien verifica un cierre sea distinto de quien lo remedió) es una regla de **proceso**, no algo que el modelo de datos garantice por sí solo. Se puede agregar el campo `verified_by` , pero que efectivamente lo llene una persona distinta del responsable de la remediación es una decisión operativa del equipo, no una brecha técnica que este documento pueda cerrar.

7. Seguridad del propio sistema — fuera del dominio del tracking, pero fundamental

Todo lo anterior asume que la plataforma que **gestiona** vulnerabilidades de terceros está, ella misma, bien asegurada. Es un dominio distinto (seguridad de la aplicación, no gestión de vulnerabilidades de activos externos), pero se incluye acá porque es un concepto de base de ciberseguridad — CIS Control 8 (Audit Log Management) y OWASP ASVS V7 (Error Handling and Logging) — que no queda cubierto por ninguna de las secciones anteriores.

7.1 No hay registro de eventos de autenticación

`AuthService.login()` (`AuthService.java:23-41`) no deja ningún rastro de los intentos de acceso: ni de los exitosos ni de los fallidos. No hay quién, cuándo, desde dónde, ni resultado. Esto es un registro distinto del Componente F (Sección 1.1) — aquel es sobre transiciones de casos de vulnerabilidad; este es sobre accesos a la plataforma misma — y ambos son necesarios, no intercambiables.

Alcance: backend.

7.2 Sin protección contra fuerza bruta en el login

No hay rate limiting ni bloqueo temporal tras varios intentos fallidos sobre `/api/auth/login` (única ruta pública además de webhooks y docs, según `SecurityConfig.java:67`). Un atacante puede probar credenciales sin fricción ni límite.

Alcance: backend.

Lo que ya está bien resuelto y conviene no tocar: `AuthService` usa BCrypt, devuelve el mismo mensaje de error para usuario inexistente y contraseña incorrecta (evita enumeración de usuarios), y el control de acceso por rol es real — `@PreAuthorize("hasRole(...)")` / `hasAnyRole(...)` está aplicado de forma consistente en prácticamente todos los controllers (`AssetController` , `UserController` , `VulnerabilityAuditController` , `EnvironmentController` , etc.). Esta sección señala dos puntos puntuales, no una debilidad general de la autenticación.